DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment No.: 2

Student Name: Probal Dhali **UID:** 24BAI71013
Branch: AIT-CSE (AI ML) **Section/Group:** B
Semester: 5TH **Date of Performance:**
Subject Name: Competitive Coding – II **Subject Code:** 24CSP-305

Experiment No: 2.1

Aim: Search Insert Position

Objective:

1. To find the target element in a sorted array.
2. To determine the correct insertion position if the target is not present.
3. To implement the Binary Search algorithm.
4. To achieve an efficient solution with logarithmic time complexity.
5. To understand searching operations in sorted arrays.

Problem Statement:

Given a sorted array of distinct integers and a target value, return its index if found. Otherwise, return the index where it should be inserted.

LeetCode Link: https://leetcode.com/problems/search-insert-position/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Set `left = 0` and `right = n - 1`.
2. Find the middle index.
3. Compare the middle element with the target.
4. Adjust the search range accordingly.
5. Return the target index or insertion position.

Code:

```
class Solution {

public:

    int searchInsert(vector<int>& nums, int target) {

        int left = 0;

        int right = nums.size() - 1;

        while (left <= right) {

            int mid = left + (right - left) / 2;

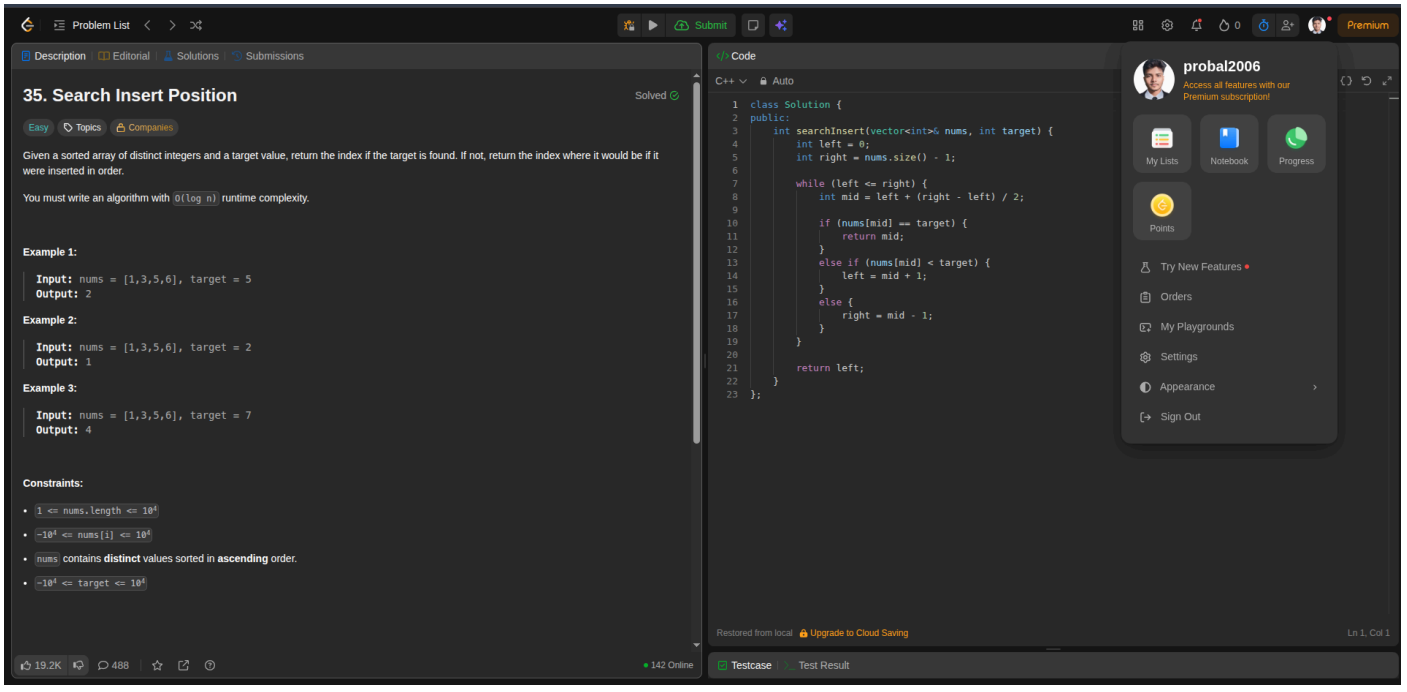
            if (nums[mid] == target) {

                return mid;

            }
```

```
        else if (nums[mid] < target) {
            left = mid + 1;
        }
        else {
            right = mid - 1;
        }
    }
    return left;
  }
};
```

Output:



Observations/Outcome: The program successfully finds the target element or returns its correct insertion position.

Time Complexity: O(log n)

Space Complexity: O(1)

Learning outcomes :

1. Learned the Binary Search algorithm.
2. Learned how to search efficiently in a sorted array.
3. Learned how to find an insertion position.
4. Understood logarithmic time complexity.
5. Improved problem-solving skills using arrays.

DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment No: 2.2

Aim: Search in a rotated sorted array

Objective:

1. To understand searching in a rotated sorted array.
2. To implement a modified Binary Search algorithm.
3. To identify which half of the array is sorted.
4. To efficiently find the target element.

Problem Statement:

Given a rotated sorted array of distinct integers and a target value, return the target's index if it exists; otherwise, return `-1`.

LeetCode Link: https://leetcode.com/problems/search-in-rotated-sorted-array/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Set `left = 0` and `right = n - 1`.
2. Find the middle element.
3. Check whether the left or right half is sorted.
4. Determine whether the target lies in the sorted half.
5. Adjust `left` or `right` accordingly.
6. Return the target index; otherwise, return `-1`.

Code:

```
class Solution {

public:

    int search(vector<int>& nums, int target) {

        int left = 0;

        int right = nums.size() - 1;

        while (left <= right) {

            int mid = left + (right - left) / 2;

            // Target found

            if (nums[mid] == target) {

                return mid;

            }

            // Left half is sorted

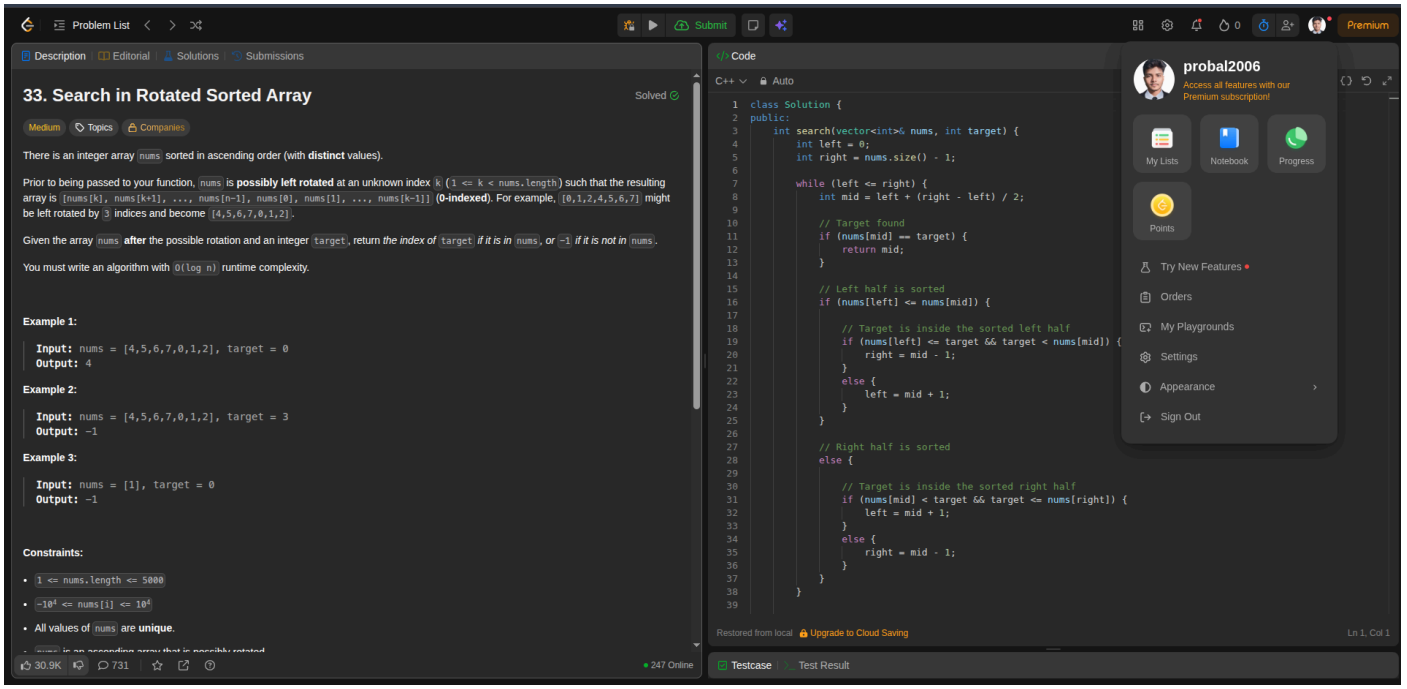
            if (nums[left] <= nums[mid]) {

                // Target is inside the sorted left half

                if (nums[left] <= target && target < nums[mid]) {
```

```
                right = mid - 1;
            }
            else {
                left = mid + 1;
            }
        }
        // Right half is sorted
        else {
            // Target is inside the sorted right half
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
    }
    return -1;
  }
};
```

Output:



Observations/Outcome:

The program successfully finds the target element in a rotated sorted array using a modified Binary Search algorithm.

Time Complexity: O(log n)

Space Complexity: O(1)

Learning outcomes:

1. Learned how Binary Search works on a rotated sorted array.
2. Learned how to identify the sorted half of an array.
3. Learned how to adapt binary search for complex problems.
4. Improved understanding of array searching techniques.
5. Learned to analyze `O(log n)` time complexity.